🏋️ FitNova Gym

Sistema Web de Gestión Integral para Gimnasios

Proyecto Integrador

Documentación Técnica — El Artefacto

Asignatura: Programación Web III

Curso: 3er Año de Sistemas

Modalidad: En parejas (dúos)

Docente: _____

Integrantes: _____ y _____

Fecha: 17/09/2026

Stack: PHP (Laravel) · MySQL/MariaDB · Blade · Tailwind · JavaScript

Índice

1. Definición del negocio
2. Ingeniería de requerimientos
3. Arquitectura de software
4. Stack tecnológico y buenas prácticas

1. Definición del Negocio

1.1 Nombre de la empresa/emprendimiento

FitNova Gym es un gimnasio local que ofrece musculación, clases grupales, pileta y entrenamiento personalizado. Hoy lleva su administración en planillas de Excel y papel, lo que genera errores, pérdida de historial y mora no controlada.

1.2 Modelo de negocio (Business Model Canvas)

Modelo Canvas		
Socios clave	**Actividades clave**	**Propuesta de valor**
Entrenadores certificados, proveedores de equipamiento, servicios contables.	Dictado de clases, planificación de rutinas, cobro de cuotas, mantenimiento del predio.	Digitalización total de la gestión: socios, membresías, pagos, rutinas y clases en una sola plataforma, con reportes en tiempo real.
Relaciones con clientes	**Canales**	**Segmento de clientes**
Atención personalizada, seguimiento de morosos y recordatorios de vencimiento.	Atención presencial en recepción y plataforma web accesible desde cualquier dispositivo.	Jóvenes y adultos de la comunidad local (16-65 años) que buscan acondicionamiento físico.
Recursos clave	**Estructura de costos**	**Fuentes de ingresos**
Instalaciones, equipamiento, personal capacitado, sistema informático.	Sueldos, servicios, alquiler, mantenimiento de equipos y hosting del sistema.	Cuotas mensuales de membresías, planes trimestrales/anuales, planes dúo y clases grupales.

1.3 Propuesta de valor

FitNova es innovador porque resuelve el **problema real** de la gestión manual: **¿qué necesidad resuelve?** Controlar de forma confiable los socios, sus vencimientos y los cobros, evitando pérdida de ingresos por mora. **¿Qué vende?** Un servicio de gestión digital integral para el gimnasio. **¿A quién va dirigido?** Al administrador (dueño), a los entrenadores y a los socios, cada uno con su propio rol y vistas. Es **necesario** porque automatiza reportes, mantiene el historial y da una experiencia moderna y responsable al socio.

2. Ingeniería de Requerimientos

2.1 Requerimientos Funcionales (RF)

ID	Nombre	Descripción
RF-01	Autenticación de usuarios	Registro, inicio y cierre de sesión con contraseñas encriptadas, validación y manejo de sesiones (sistema nativo de Laravel/Breeze).

RF-02	Roles y permisos	Tres roles con vistas y acciones diferenciadas: administrador, entrenador y socio (middleware de rol).
RF-03	Gestión de socios	Alta, edición, baja y búsqueda de socios (nombre, email, documento) con historial completo.
RF-04	Gestión de planes	CRUD de planes de membresía con precio, duración y estado.
RF-05	Asignación de membresías	Asociar un plan a un socio calculando automáticamente la fecha de vencimiento.
RF-06	Registro de pagos	Cobro de cuotas con monto, método (efectivo/tarjeta/transferencia) y reactivación de membresías vencidas.
RF-07	Control de mora	Detección automática de membresías vencidas y alertas en el panel principal.
RF-08	Gestión de rutinas	Entrenadores asignan rutinas a socios con ejercicios parametrizados (series, repeticiones, peso).
RF-09	Agenda de clases	Alta de clases grupales con cupo, fecha, hora y entrenador a cargo.
RF-10	Inscripción a clases	El socio se inscribe/cancela en clases verificando cupo; el entrenador marca asistencia.
RF-11	Dashboard y reportes	Métricas (ingresos, socios activos, morosos), gráficos y exportación de reportes a CSV.
RF-12	Búsqueda asincrónica	Búsqueda de socios en tiempo real mediante JavaScript (Fetch) contra una API JSON.

2.2 Requerimientos No Funcionales (RNF)

ID	Categoría	Especificación
RNF-01	Rendimiento	Tiempo de respuesta menor a 2 segundos por página en condiciones normales de uso local.
RNF-02	Responsividad	Diseño 100% responsivo (Tailwind CSS) adaptado a desktop, tablet y móvil.
RNF-03	Seguridad	Contraseñas con hash bcrypt, protección CSRF en formularios y protección de rutas por rol.
RNF-04	Compatibilidad	Compatibilidad con navegadores modernos (Chrome, Firefox, Edge) y versiones recientes de PHP 8.2+.
RNF-05	Usabilidad	Interfaz en español, mensajes de éxito/error claros y validación de formularios en el servidor.
RNF-06	Mantenibilidad	Código limpio y modular (controladores, modelo, Form Requests), formateado con Laravel Pint.
RNF-07	Disponibilidad	Funcionamiento continuo en el entorno XAMPP; instrucciones de instalación documentadas en README.
RNF-08	Integridad de datos	Integridad referencial con claves foráneas, migraciones versionadas y sembradores reproducibles.

3. Arquitectura de Software

3.1 Arquitectura Modelo-Vista-Controlador (MVC) en Laravel

Laravel implementa el patrón MVC de forma nativa y similar a otros frameworks PHP:

- **Modelo (Models):** clases Eloquent que representan cada tabla (por ejemplo `User`, `Plan`, `Membresia`, `Pago`). Contienen la lógica de negocio, relaciones y consultas.
- **Vista (Views Blade):** plantillas HTML renderizadas con Blade, con componentes reutilizables y Tailwind CSS. Las vistas reciben datos ya preparados por el controlador.
- **Controlador (Controllers):** orquestan la petición: reciben el `Request`, validan con Form Requests, interactúan con los modelos (Eloquent ORM), y devuelven vistas o redirecciones.
- **Rutas (routes/web.php):** el front controller centraliza la entrada de cada URL y aplica middleware de autenticación y de rol.

3.2 Flujo de datos entre Front-end y Back-end

1. El navegador solicita una URL (ej. `GET /socios`).
2. El **routing** resuelve la ruta y aplica middleware (`auth`, `verified`, `role`).
3. El **controlador** recibe la petición, valida con la Form Request y consulta los **modelos Eloquent**.
4. Eloquent traduce las consultas a SQL sobre **MySQL/MariaDB** mediante migraciones y devuelve colecciones de objetos.
5. El controlador pasa los datos a la **vista Blade**, que genera el HTML con Tailwind.
6. Para dinamismo, el **JavaScript (Fetch)** consulta endpoints JSON (ej. búsqueda de socios) y actualiza el DOM sin recargar la página.
7. Los formularios envían datos con token **CSRF**; los controladores procesan, guardan y redirigen con mensajes flash de sesión.

> **Flujo resumido:** Routes → Middleware → Controller → Form Request (validación) → Model (Eloquent) → MySQL ⇄ Model → Controller → View (Blade/Tailwind) → HTML/JS.

3.3 Modelado de la Base de Datos (DER optimizado para MySQL)

La base `gimnasio_db` se crea con migraciones de Laravel y se siembra con dados demo. Las claves primarias se marcan en `rojo` y las foráneas en `verde`.



socio_id (FK→users)

plan_id (FK→planes)

fecha_inicio

fecha_fin

estado (activa/vencida)

pagos

id (PK)

membresia_id (FK→membresias)

socio_id (FK→users)

monto

fecha_pago

metodo

rutinas

id (PK)

entrenador_id (FK→users)

socio_id (FK→users)

titulo

fecha_asignacion

estado

ejercicios

id (PK)

rutina_id (FK→rutinas)

nombre

series, repeticiones

peso, orden

clases

id (PK)

entrenador_id (FK→users)

nombre

fecha, hora

cupo

asistencias

id (PK)

clase_id (FK→clases)

socio_id (FK→users)

fecha

asistio (bool)

Cardinalidad de las relaciones

Relación	Tipo	Tabla puente / Clave foránea
users (socio) → membresias	1:N	membresias.socio_id
planes → membresias	1:N	membresias.plan_id
membresias → pagos	1:N	pagos.membresia_id
users (entrenador) → rutinas	1:N	rutinas.entrenador_id
users (socio) → rutinas	1:N	rutinas.socio_id
rutinas → ejercicios	1:N	ejercicios.rutina_id
users (entrenador) → clases	1:N	clases.entrenador_id
clases → asistencias	1:N	asistencias.clase_id
users (socio) → asistencias	1:N	asistencias.socio_id

4. Stack Tecnológico y Buenas Prácticas

Componente	Tecnología
Entorno de desarrollo	XAMPP (Apache, PHP 8.2, MariaDB/MySQL)
Back-end	PHP 8.2 · Framework Laravel 12 (rutas, Eloquent ORM, migraciones, validación, sesiones)
Autenticación	Laravel Breeze (login, registro, olvido de contraseña, confirmación)
Front-end	Blade · Tailwind CSS · JavaScript (Fetch) · Chart.js · Vite
Base de datos	MySQL/MariaDB con migraciones y sembradores (Seeders + Factories)
Calidad de código	Laravel Pint · Tests automatizados (PHPUnit / Pest derivado)
Control de versiones	Git + GitHub con README de instalación

Buenas prácticas aplicadas

- Uso correcto de variables de entorno en `.env` (nunca se suben credenciales al repositorio).
- Código limpio y modular: controladores ligeros, lógica de negocio en modelos, validación en Form Requests.
- Sembradores (Seeders/Factories) que reproducen datos de prueba para demostrar el sistema.
- Control de versiones con commits por fase y historial compartido del dúo.
- Integridad referencial de la base de datos garantizada por migraciones con claves foráneas.

Usuarios de demostración (password: `password`)

Rol	Email
Administrador	admin@fitnova.com
Entrenador	entrenador@fitnova.com
Socio	socio@fitnova.com

Repositorio y ejecución: el código fuente se encuentra en el repositorio Git del proyecto. Para reproducir el sistema se siguen las instrucciones del archivo `README.md` : clonar, instalar dependencias con Composer y npm, configurar `.env` , ejecutar migraciones y sembrar la base.